

```

/*211, Ivan Ciller Lopez, Mohamed Rida Chahdaoui Moujib, */
/* 100522245@alumnos.uc3m.es, 100522202@alumnos.uc3m.es*/

%{
    // SECCION 1 Declaraciones de C-Yacc

#include <stdio.h>
#include <ctype.h>
#include <string.h>
#include <stdlib.h>

#define FF fflush(stdout);

int yylex () ;
int yyerror (char *mensaje) ;
char *my_malloc (int) ;
char *gen_code (char *) ;
char *int_to_string (int) ;
char *char_to_string (char) ;

char *escape_lisp_string (char *) ;
char *next_printf_arg (char **) ;
void add_printf_piece (char *, int *, const char *) ;
char *translate_printf (char *, char *) ;

char temp [2048] ;

/* Gestión de variables locales para el Punto 8 */
char *local_vars[100];
int n_locals = 0;
char *current_func = "main";

void add_local(char *name) {
    local_vars[n_locals++] = gen_code(name);
}

char* transform_id(char *name) {
    for(int i = 0; i < n_locals; i++) {
        if(strcmp(local_vars[i], name) == 0) {
            sprintf(temp, "%s_%s", current_func, name);
            return gen_code(temp);
        }
    }
    return name;
}

typedef struct ASTnode t_node ;

struct ASTnode {
    char *op ;
    int type ;
    t_node *left ;
    t_node *right ;
} ;

typedef struct s_attr {
    int value ;
    char *code ;
    t_node *node ;
} ;

```

```

} t_attr ;

#define YYSTYPE t_attr

%}

%token NUMBER
%token IDENTIF
%token INTEGER
%token STRING
%token MAIN
%token WHILE
%token IF ELSE
%token FOR SWITCH CASE DEFAULT BREAK RETURN INC DEC
%token PRINTF PUTS

/* Definición de Tokens para Operadores Lógicos y Relacionales */
%token AND OR NOT IGUAL DIFERENTE MENOR_IGUAL MAYOR_IGUAL

/* Tabla de Precedencia y Asociatividad */
%right '='
%left OR
%left AND
%left IGUAL DIFERENTE
%left '<' '>' MENOR_IGUAL MAYOR_IGUAL
%left '+' '-'
%left '*' '/' '%'
%left UNARY_SIGN NOT

%%

axioma:
    top_level_list funcion_main          { ; }
;

top_level_list:
    /* lambda */
    | declaracion ';'
    | funcion
    { printf ("%s\n", $1.code) ; } top_level_list
    { printf ("%s\n", $1.code) ; } top_level_list
;

funcion:
    INTEGER IDENTIF '(' { current_func = gen_code($2.code); n_locals = 0; } parametros ')' '{' declaraciones_locales lista_sentencias '}' {
        if (strlen($5.code) == 0)
            sprintf (temp, "(defun %s ()\n%s\n%s\n)", $2.code, $8.code, $9.code) ;
        else
            sprintf (temp, "(defun %s (%s)\n%s\n%s\n)", $2.code, $5.code, $8.code, $9.code) ;
        $$code = gen_code (temp) ;
    }
    | IDENTIF '(' { current_func = gen_code($1.code); n_locals = 0; } parametros ')' '{' declaraciones_locales lista_sentencias '}' {
        if (strlen($4.code) == 0)
            sprintf (temp, "(defun %s ()\n%s\n%s\n)", $1.code, $7.code, $8.code) ;
        else
            sprintf (temp, "(defun %s (%s)\n%s\n%s\n)", $1.code, $4.code, $7.code, $8.code) ;
        $$code = gen_code (temp) ;
    }
;

parametros:

```

```

/* lambda */
| lista_parametros

;

lista_parametros:
    parametro
| parametro ',' lista_parametros

;

parametro:
    INTEGER IDENTIF

| IDENTIF

;

/* Declaraciones globales ahora son parte de top_level_list */

declaraciones_locales:
    /* lambda */
| declaracion_local ';' declaraciones_locales

;

declaracion:
    INTEGER lista_vars

;

/* Declaración local específica para concatenar con main_ */
declaracion_local:
    INTEGER lista_vars_locales

;

lista_vars:
    vars
| vars ',' lista_vars

;

lista_vars_locales:
    vars_locales
| vars_locales ',' lista_vars_locales

;

```

```

{ $$code = gen_code ("" ) ; }
{ $$code = $1.code ; }

{ $$code = $1.code ; }
{
    sprintf (temp, "%s %s", $1.code, $3.code) ;
    $$code = gen_code (temp) ;
}

{
    add_local($2.code);
    sprintf(temp, "%s_%s", current_func, $2.code);
    $$code = gen_code (temp) ;
}

{
    add_local($1.code);
    sprintf(temp, "%s_%s", current_func, $1.code);
    $$code = gen_code (temp) ;
}

{ $$code = gen_code ("" ) ; }
{
    sprintf (temp, "%s%s", $1.code, $3.code) ;
    $$code = gen_code (temp) ;
}

{ $$code = $2.code ; }

{ $$code = $2.code ; }

{ $$code = $1.code ; }
{
    sprintf (temp, "%s\n%s", $1.code, $3.code) ;
    $$code = gen_code (temp) ;
}

{ $$code = $1.code ; }
{
    sprintf (temp, "%s\n%s", $1.code, $3.code) ;
    $$code = gen_code (temp) ;
}

```

```

vars:
    IDENTIF
        {
            sprintf (temp, "(setq %s 0)", $1.code) ;
            $$code = gen_code (temp) ;
        }
    | IDENTIF '=' cte
        {
            sprintf (temp, "(setq %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    | IDENTIF '[' cte ']'
        {
            sprintf (temp, "(setq %s (make-array %s))", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }
;

/* Punto 8: Registro de variables locales y prefijo main_ */
vars_locales:
    IDENTIF
        {
            add_local ($1.code) ;
            sprintf (temp, "(setq %s_%s 0)", current_func, $1.code) ;
            $$code = gen_code (temp) ;
        }
    | IDENTIF '=' cte
        {
            add_local ($1.code) ;
            sprintf (temp, "(setq %s_%s %s)", current_func, $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    | IDENTIF '[' cte ']'
        {
            add_local ($1.code) ;
            sprintf (temp, "(setq %s_%s (make-array %s))", current_func, $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }
;

cte:
    NUMBER
        {
            sprintf (temp, "%d", $1.value) ;
            $$code = gen_code (temp) ;
        }
    | '+' NUMBER %prec UNARY_SIGN
        {
            sprintf (temp, "%d", $2.value) ;
            $$code = gen_code (temp) ;
        }
    | '-' NUMBER %prec UNARY_SIGN
        {
            sprintf (temp, "-%d", $2.value) ;
            $$code = gen_code (temp) ;
        }
    }
;

/* CORRECCION:
   - Ya NO se añade (main) automáticamente
   - Se imprime la definición de main directamente aquí
*/
funcion_main:
    MAIN { current_func = "main"; n_locals = 0; } '(' ')' '{' declaraciones_locales lista_sentencias '}' {
        sprintf (temp, "(defun main ()\n%s\n%s\n)", $6.code, $7.code) ;
        $$code = gen_code (temp) ;
        printf ("%s\n", $$code) ;
    }

```

```

;

lista_sentencias:
    /* lambda */
    | sentencia ';' lista_sentencias

    | sentencia_control lista_sentencias

;

/* CORRECCION:
    lista_argumentos para printf ahora guarda argumentos "crudos"
    y acepta tanto expresiones como STRING
*/
lista_argumentos:
    argumento_printf
    | argumento_printf ',' lista_argumentos

;

argumento_printf:
    expresion
    | STRING

;

lista_expr:
    /* lambda */
    | expresion
    | expresion ',' lista_expr

;

sentencia:
    IDENTIF '=' expresion

    | IDENTIF '[' expresion ']' '=' expresion

```

```

}

{ $.code = gen_code ("") ; }
{
if (strlen ($3.code) == 0)
    sprintf (temp, "    %s", $1.code) ;
else
    sprintf (temp, "    %s\n%s", $1.code, $3.code) ;
$.code = gen_code (temp) ;
}

{
if (strlen ($2.code) == 0)
    sprintf (temp, "    %s", $1.code) ;
else
    sprintf (temp, "    %s\n%s", $1.code, $2.code) ;
$.code = gen_code (temp) ;
}

{ $.code = $1.code ; }
{
sprintf (temp, "%s\037%s", $1.code, $3.code) ;
$.code = gen_code (temp) ;
}

{ $.code = $1.code ; }
{
char *esc = escape_lisp_string ($1.code) ;
sprintf (temp, "\"%s\"", esc) ;
$.code = gen_code (temp) ;
}

{ $.code = gen_code ("") ; }
{ $.code = $1.code ; }
{
sprintf (temp, "%s %s", $1.code, $3.code) ;
$.code = gen_code (temp) ;
}

{
char *final_id = transform_id ($1.code) ;
sprintf (temp, "(setf %s %s)", final_id, $3.code) ;
$.code = gen_code (temp) ;
}

{
char *final_id = transform_id ($1.code) ;

```

```

        sprintf (temp, "(setf (aref %s %s) %s)", final_id, $3.code, $6.code) ;
        $$code = gen_code (temp) ;
    }
    | RETURN expression
    {
        sprintf (temp, "(return-from %s %s)", current_func, $2.code) ;
        $$code = gen_code (temp) ;
    }
    | PUTS '(' STRING ')'
    {
        char *esc = escape_lisp_string ($3.code) ;
        sprintf (temp, "(print \"%s\")", esc) ;
        $$code = gen_code (temp) ;
    }
    | PRINTF '(' STRING ')'
    {
        $$code = translate_printf ($3.code, "") ;
    }
    | PRINTF '(' STRING ',' lista_argumentos ')'
    {
        $$code = translate_printf ($3.code, $5.code) ;
    }
    | IDENTIF '(' lista_expr ')'
    {
        if (strlen($3.code) == 0)
            sprintf (temp, "(%s)", $1.code) ;
        else
            sprintf (temp, "(%s %s)", $1.code, $3.code) ;
        $$code = gen_code (temp) ;
    }
;

sentencia_control:
    WHILE '(' expresion ')' '{' lista_sentencias '}'
    {
        sprintf (temp, "(loop while %s do\n%s\n    )", $3.code, $6.code) ;
        $$code = gen_code (temp) ;
    }
    | IF '(' expresion ')' '{' lista_sentencias '}'
    {
        sprintf (temp, "(if %s (progn\n%s\n    ))", $3.code, $6.code) ;
        $$code = gen_code (temp) ;
    }
    | IF '(' expresion ')' '{' lista_sentencias '}' ELSE '{' lista_sentencias '}' {
        sprintf (temp, "(if %s (progn\n%s\n    ) (progn\n%s\n    ))", $3.code, $6.code, $10.code) ;
        $$code = gen_code (temp) ;
    }
    | FOR '(' for_init ';' expresion ';' for_step ')' '{' lista_sentencias '}' {
        sprintf (temp, "(progn\n    %s\n    (loop while %s do\n%s\n    %s\n    ))", $3.code, $5.code, $10.code, $7.code) ;
        $$code = gen_code (temp) ;
    }
    | SWITCH '(' expresion ')' '{' lista_cases '}'
    {
        sprintf (temp, "(case %s\n%s\n    )", $3.code, $6.code) ;
        $$code = gen_code (temp) ;
    }
;

for_init:
    IDENTIF '=' expresion
    {
        char *final_id = transform_id ($1.code) ;
        sprintf (temp, "(setf %s %s)", final_id, $3.code) ;
        $$code = gen_code (temp) ;
    }
;

for_step:

```

<pre> INC '(' IDENTIF ')' DEC '(' IDENTIF ')' ; lista_cases: case_block case_block lista_cases ; case_block: CASE cte ':' lista_sentencias BREAK ';' DEFAULT ':' lista_sentencias DEFAULT ':' lista_sentencias BREAK ';' ; expresion: expr_or ; expr_or: expr_and expr_or OR expr_and ; expr_and: expr_igualdad expr_and AND expr_igualdad ; expr_igualdad: expr_relacional expr_igualdad IGUAL expr_relacional </pre>	<pre> { char *final_id = transform_id (\$3.code) ; sprintf (temp, "(setf %s (+ %s 1))", final_id, final_id) ; \$\$code = gen_code (temp) ; } { char *final_id = transform_id (\$3.code) ; sprintf (temp, "(setf %s (- %s 1))", final_id, final_id) ; \$\$code = gen_code (temp) ; } { \$\$code = \$1.code ; } { sprintf (temp, "%s\n%s", \$1.code, \$2.code) ; \$\$code = gen_code (temp) ; } { sprintf (temp, " (%s (progn\n%s\n))", \$2.code, \$4.code) ; \$\$code = gen_code (temp) ; } { sprintf (temp, " (otherwise (progn\n%s\n))", \$3.code) ; \$\$code = gen_code (temp) ; } { sprintf (temp, " (otherwise (progn\n%s\n))", \$3.code) ; \$\$code = gen_code (temp) ; } { \$\$ = \$1 ; } { { sprintf (temp, "(or %s %s)", \$1.code, \$3.code) ; \$\$code = gen_code (temp) ; } } { \$\$ = \$1 ; } { sprintf (temp, "(and %s %s)", \$1.code, \$3.code) ; \$\$code = gen_code (temp) ; } { \$\$ = \$1 ; } { sprintf (temp, "(= %s %s)", \$1.code, \$3.code) ; </pre>
---	---

```

    | expr_igualdad DIFERENTE expr_relacional
    {
        $$code = gen_code (temp) ;
        {
            sprintf (temp, "(/= %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }

;

expr_relacional:
    expr_aditiva
    | expr_relacional '>' expr_aditiva
    {
        { $$ = $1 ; }
        {
            sprintf (temp, "(> %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }

    | expr_relacional '<' expr_aditiva
    {
        {
            sprintf (temp, "(< %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }

    | expr_relacional MAYOR_IGUAL expr_aditiva
    {
        {
            sprintf (temp, "(>= %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }

    | expr_relacional MENOR_IGUAL expr_aditiva
    {
        {
            sprintf (temp, "(<= %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }

;

expr_aditiva:
    expr_multiplicativa
    | expr_aditiva '+' expr_multiplicativa
    {
        { $$ = $1 ; }
        {
            sprintf (temp, "(+ %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }

    | expr_aditiva '-' expr_multiplicativa
    {
        {
            sprintf (temp, "(- %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }

;

expr_multiplicativa:
    expr_unaria
    | expr_multiplicativa '*' expr_unaria
    {
        { $$ = $1 ; }
        {
            sprintf (temp, "(* %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }

    | expr_multiplicativa '/' expr_unaria
    {
        {
            sprintf (temp, "(/ %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }

    | expr_multiplicativa '%' expr_unaria
    {
        {
            sprintf (temp, "(mod %s %s)", $1.code, $3.code) ;
            $$code = gen_code (temp) ;
        }
    }

;

expr_unaria:
    termino
    | '+' termino %prec UNARY_SIGN
    {
        { $$ = $1 ; }
        { $$ = $2 ; }
    }

```



```

| '-' termino %prec UNARY_SIGN
{
    sprintf (temp, "(- %s)", $2.code) ;
    $$code = gen_code (temp) ;
}

| NOT expr_unaria
{
    sprintf (temp, "(not %s)", $2.code) ;
    $$code = gen_code (temp) ;
}

;

termino:
    operando
{
    { $$ = $1 ; }

operando:
    IDENTIF
{
    char *final_id = transform_id ($1.code) ;
    sprintf (temp, "%s", final_id) ;
    $$code = gen_code (temp) ;
}

| NUMBER
{
    sprintf (temp, "%d", $1.value) ;
    $$code = gen_code (temp) ;
}

| IDENTIF '[' expresion ']'
{
    char *final_id = transform_id ($1.code) ;
    sprintf (temp, "(aref %s %s)", final_id, $3.code) ;
    $$code = gen_code (temp) ;
}

| IDENTIF '(' lista_expr ')'
{
    {
        if (strlen($3.code) == 0)
            sprintf (temp, "(%s)", $1.code) ;
        else
            sprintf (temp, "(%s %s)", $1.code, $3.code) ;
        $$code = gen_code (temp) ;
    }
}

| '(' expresion ')'
{
    { $$ = $2 ; }

;

%%
// SECCION 4    Codigo en C

int n_line = 1 ;

int yyerror (char *mensaje)
{
    fprintf (stderr, "%s en la linea %d\n", mensaje, n_line) ;
    return 0 ;
}

char *gen_code (char *name)
{
    char *p ;
    int l = strlen (name) + 1 ;
    p = (char *) my_malloc (l) ;
    strcpy (p, name) ;
    return p ;
}

char *my_malloc (int nbytes)

```

```

{
    char *p ;
    p = (char *) malloc (nbytes) ;
    if (p == NULL) {
        fprintf (stderr, "No queda memoria\n") ;
        exit (0) ;
    }
    return p ;
}

char *escape_lisp_string (char *src)
{
    char buffer[4096] ;
    int i = 0, j = 0 ;

    while (src[i] != '\0' && j < 4090) {
        if (src[i] == '"' || src[i] == '\\')
            buffer[j++] = '\\';
        buffer[j++] = src[i++] ;
    }
    buffer[j] = '\0' ;
    return gen_code (buffer) ;
}

char *next_printf_arg (char **cursor)
{
    char buffer[4096] ;
    int i = 0 ;

    if (cursor == NULL || *cursor == NULL || **cursor == '\0')
        return NULL ;

    while (**cursor != '\0' && **cursor != '\037' && i < 4095) {
        buffer[i++] = **cursor ;
        (*cursor)++ ;
    }

    buffer[i] = '\0' ;

    if (**cursor == '\037')
        (*cursor)++ ;

    return gen_code (buffer) ;
}

void add_printf_piece (char *dest, int *npieces, const char *piece)
{
    if (piece == NULL || piece[0] == '\0')
        return ;

    if (*npieces == 0)
        strcpy (dest, piece) ;
    else {
        strcat (dest, "\n " ) ;
        strcat (dest, piece) ;
    }

    (*npieces)++ ;
}

```

```

char *translate_printf (char *format, char *args)
{
    char result[16384] ;
    char piece[8192] ;
    char final_code[20000] ;
    char *cursor ;
    char *arg ;
    int npieces = 0 ;
    (void) format ;

    result[0] = '\0' ;
    cursor = args ;

    while ((arg = next_printf_arg (&cursor)) != NULL) {
        sprintf (piece, "(princ %s)", arg) ;
        add_printf_piece (result, &npieces, piece) ;
    }

    if (npieces == 0)
        return gen_code ("") ;

    if (npieces == 1)
        return gen_code (result) ;

    sprintf (final_code, "(progn\n    %s\n)", result) ;
    return gen_code (final_code) ;
}

typedef struct s_keyword {
    char *name ;
    int token ;
} t_keyword ;

t_keyword keywords [] = {
    "main",    MAIN,
    "printf",  PRINTF,
    "puts",    PUTS,
    "int",     INTEGER,
    "while",   WHILE,
    "if",      IF,
    "else",    ELSE,
    "for",     FOR,
    "switch",  SWITCH,
    "case",    CASE,
    "default", DEFAULT,
    "break",   BREAK,
    "return",  RETURN,
    "inc",     INC,
    "dec",     DEC,
    "&&",      AND,
    "||",      OR,
    "==",      IGUAL,
    "!=",      DIFERENTE,
    "<=",      MENOR_IGUAL,
    ">=",      MAYOR_IGUAL,
    "!",       NOT,
    NULL,     0
};

```

```

t_keyword *search_keyword (char *symbol_name)
{
    int i = 0;
    while (keywords[i].name != NULL) {
        if (strcmp (keywords[i].name, symbol_name) == 0)
            return &keywords[i];
        i++;
    }
    return NULL;
}

int yylex ()
{
    // NO MODIFICAR ESTA FUNCION SIN PERMISO
    int i ;
    unsigned char c ;
    unsigned char cc ;
    char ops_expandibles [] = "!<=|>%&/+.*" ;
    char temp_str [256] ;
    t_keyword *symbol ;

    do {
        c = getchar () ;

        if (c == '#') {
            do {
                c = getchar () ;
            } while (c != '\n') ;
        }

        if (c == '/') {
            cc = getchar () ;
            if (cc != '/') {
                ungetc (cc, stdin) ;
            } else {
                c = getchar () ;
                if (c == '@') {
                    do {
                        c = getchar () ;
                        putchar (c) ;
                    } while (c != '\n') ;
                } else {
                    while (c != '\n') {
                        c = getchar () ;
                    }
                }
            }
        }
        } else if (c == '\\') c = getchar () ;

        if (c == '\n')
            n_line++ ;

    } while (c == ' ' || c == '\n' || c == 10 || c == 13 || c == '\t') ;

    if (c == '"') {
        i = 0 ;
        do {
            c = getchar () ;

```

```

        temp_str [i++] = c ;
    } while (c != '\"' && i < 255) ;
    if (i == 256) {
        printf ("AVISO: string con mas de 255 caracteres en linea %d\n", n_line) ;
    }
    temp_str [--i] = '\0' ;
    yylval.code = gen_code (temp_str) ;
    return (STRING) ;
}

if (c == '.' || (c >= '0' && c <= '9')) {
    ungetc (c, stdin) ;
    scanf ("%d", &yylval.value) ;
    return NUMBER ;
}

if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {
    i = 0 ;
    while (((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') ||
        (c >= '0' && c <= '9') || c == '_') && i < 255) {
        temp_str [i++] = tolower (c) ;
        c = getchar () ;
    }
    temp_str [i] = '\0' ;
    ungetc (c, stdin) ;

    yylval.code = gen_code (temp_str) ;
    symbol = search_keyword (yylval.code) ;
    if (symbol == NULL) {
        return (IDENTIF) ;
    } else {
        return (symbol->token) ;
    }
}

if (strchr (ops_expandibles, c) != NULL) {
    cc = getchar () ;
    sprintf (temp_str, "%c%c", (char) c, (char) cc) ;
    symbol = search_keyword (temp_str) ;
    if (symbol == NULL) {
        ungetc (cc, stdin) ;
        yylval.code = NULL ;
        return (c) ;
    } else {
        yylval.code = gen_code (temp_str) ;
        return (symbol->token) ;
    }
}

if (c == EOF || c == 255 || c == 26) {
    return (0) ;
}

return c ;
}

int main ()
{
    yyparse();
}

```

```
    return 0;  
}
```